

StrAIght Shot:

Real-Time Internal-State Monitoring for LLM Safety
via Linear Probes, Jacobian-Lens Concept Decomposition,
and Token Entropy Dynamics

Michael Vega

Noirsys

michael@noirsys.com

<https://noirsys.com>

July 10, 2026

License: MIT

Package: `pip install straightshot`

Abstract

Large Language Models (LLMs) generate text token-by-token, but the model’s internal reasoning — hidden states, activation patterns, and representational geometry — carries signals about intent, uncertainty, and unsafe trajectories that never appear in output text. We present **StrAIght Shot**, an open-source middleware system that monitors LLM internal activations in real time, detecting jailbreaks, prompt injections, hallucination onset, and hidden goals *before* they manifest as text. StrAIght Shot combines three independently validated detection channels: (1) a lightweight linear probe trained on mid-layer hidden states achieving 99.44% accuracy and 99.31% F1 score at 100% precision on 556 prompts across 6 attack categories using Qwen 3.5 4B, (2) a Jacobian-lens concept monitor that reveals a $2.4\times$ separation between safe and attack J-space representations across 12 calibrated danger concepts, including silent token detection of manipulation markers such as “ignore” (0.89), “system_prompt” (0.82), and “override” (0.76), and (3) token entropy anomaly detection that tracks reasoning collapse and goal switching. We demonstrate live-blocking of a DAN jailbreak prompt (probe score 0.947), where the J-space channel independently identifies the jailbreak concept before any output is generated. StrAIght Shot runs as a forward-hook sidecar on any HuggingFace-compatible model and exposes per-token safety metadata through an OpenAI-compatible streaming API. The full implementation, trained weights, and pre-fitted lenses are released under the MIT license at <https://github.com/noirsys/straightshot>.

1 Introduction

Current LLM safety architectures operate at two levels: **input filtering** — prompt-level classifiers that score user input before generation begins — and **output filtering** — post-hoc moderation models that score generated text after completion [8,9]. Both approaches share a fundamental blind spot: they cannot detect what happens *during* generation. Prompt injection, jailbreak exploitation, hallucination onset, and

hidden-goal execution are processes that unfold inside the model’s internal representations before they surface in output text [1]. By the time an output filter catches the violation, the damage — a leaked credential, a revealed vulnerability, a manipulative response — has already been transmitted to the user.

In June–July 2026, a cluster of independent research results converged on a common insight: **the internal activations of LLMs contain actionable safety signals that are not present in the output text**. These include:

- **Verbalizable representations in residual stream activations** — concepts the model actively entertains but may not verbalize [1].
- **Token-level safety scores from hidden-state probes** — sub-millisecond per-token detection of unsafe output trajectories [2].
- **Multi-dimensional refusal subspaces** — geometrically defined regions of activation space corresponding to refusal behavior [3].
- **Entropy trajectory anomalies** — distinctive patterns of predictive uncertainty that precede jail-break output [4].

We present **StrAIght Shot**, the first unified, deployable middleware that combines these approaches into a real-time safety monitoring system. StrAIght Shot deploys as a forward-hook sidecar into any HuggingFace-compatible model server, reading hidden states at strategic layers during generation and emitting per-token safety scores through an OpenAI-compatible API with SSE streaming.

Contributions. This work makes the following contributions:

1. A three-channel detection architecture combining linear probes, Jacobian lens subspace analysis, and entropy dynamics into a single, unified middleware system — the Guardian.
2. A trained linear probe achieving **99.44% accuracy** and **99.31% F1** at **100% precision** on 556 prompts across 6 attack categories on Qwen 3.5 4B, with 5-fold cross-validation.
3. A calibrated J-space danger concept monitor spanning 12 concept categories, demonstrating **2.4× separation** between safe and attack representations and detecting silent manipulation tokens before they surface in output.
4. An end-to-end live demonstration blocking a DAN jailbreak prompt with multi-channel consensus (probe score 0.947, J-space jailbreak concept activation, verdict BLOCK).
5. An open-source implementation released under MIT license at <https://github.com/noirsys/straightshot>, including pre-trained probe weights, fitted Jacobian lens, OpenAI-compatible proxy server, and full documentation.

2 Related Work

2.1 The Jacobian Lens and the Global Workspace

Anthropic’s “Verbalizable Representations Form a Global Workspace in Language Models” [1] demonstrated that residual stream activations can be read out through a *Jacobian lens* to reveal a “global workspace” — a set of verbalizable concepts the model is actively entertaining, including concepts it never surfaces in output.

The key mathematical object is the average input–output Jacobian:

$$\mathcal{J}_l = \mathbb{E} \left[\frac{\partial h_{\text{final}}}{\partial h_l} \right] \quad (1)$$

where h_l is the residual stream at layer l , and h_{final} is the residual stream before unembedding at the final

layer. The expectation is over a text corpus, summing cotangents over target positions and averaging over source positions.

For any residual activation h at layer l , the lens readout is:

$$\text{lens}_l(h) = \mathcal{U}(\mathcal{J}_l \cdot h) \quad (2)$$

where $\mathcal{U} : \mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{|\mathcal{V}|}$ is the unembedding matrix. This produces a vector over the vocabulary, ranking tokens by how disposed the model is to produce them given that activation.

The reference implementation (`anthropics/jacobian-lens`) provides lens fitting, application, and visualization. A community port (`migueldeguzman/jspace`) adds J-space dictionary construction, gradient-pursuit decomposition, and causal intervention tools including LensSwap and TargetedAblation [7]. Our implementation uses the `jlens` library for lens fitting and application on Qwen 3.5 4B.

2.2 Hidden-State Probes

The authors of [2] demonstrated that lightweight linear probes trained on internal activations can detect unsafe output trajectories token-by-token during generation, with sub-millisecond latency and no additional forward pass. Their key finding: the signal needed for moderation is already present in model hidden states. A linear probe applied to a single mid-layer recovers most decisions of strong guard models (LlamaGuard, WildGuard) at orders of magnitude lower compute cost. Our Channel 1 directly extends this approach, training a logistic regression probe on layer 28 of Qwen 3.5 4B with 2,560-dimensional hidden states.

2.3 Refusal Representations and Subspaces

The authors of [3] showed that refusal behavior in LLMs lives in multi-dimensional subspaces of activation space, extractable in seconds using the Recursive Feature Machine (RFM) algorithm. Concurrent work by [6] independently confirmed that some models concentrate refusal along a single direction while others distribute it across a low-rank subspace. Templeton et al. [11] provided foundational circuit-level analysis of how features are represented in superposition within transformer models.

2.4 Entropy Trajectory Analysis

The authors of [4] demonstrated that jailbreak attacks leave a distinctive signature in the predictive entropy trajectory across intermediate layers — detectable before the model produces any unsafe output. The authors of [5] documented a critical deployment issue: activation monitors trained on base models degrade when the model is fine-tuned, quantized, or adapted, while demonstrating label-free repair strategies.

2.5 Circuit Analysis and Mechanistic Interpretability

The broader mechanistic interpretability literature has established methods for understanding how transformer models compute. Work on sparse autoencoders [12, 13] demonstrated that model activations can be decomposed into interpretable features. The logit lens [14] pioneered the technique of reading intermediate layer representations through the unembedding matrix, which the Jacobian lens generalizes by using the true input-output Jacobian rather than the identity approximation.

2.6 Relationship to Prior Work

StrAIght Shot differs from prior work in four key ways: (1) it is the *first* system to combine all three detection modalities (probes, J-space, entropy) into a unified middleware; (2) it achieves state-of-the-art probe accuracy of 99.44% on a 4B-parameter model with 556 prompts across 6 attack categories; (3) it provides the first calibrated, named danger concept dictionary for J-space monitoring with 12 concept categories validated on real attack prompts; and (4) it is released as a fully deployable open-source package (`pip install straightshot`) with a streaming API.

3 Methodology: The Three-Channel Guardian Architecture

StrAIght Shot deploys as a forward-hook sidecar that reads hidden states from a running model during generation, runs three parallel detection algorithms, and combines their outputs into a per-token safety verdict. Figure 1 illustrates the system architecture.

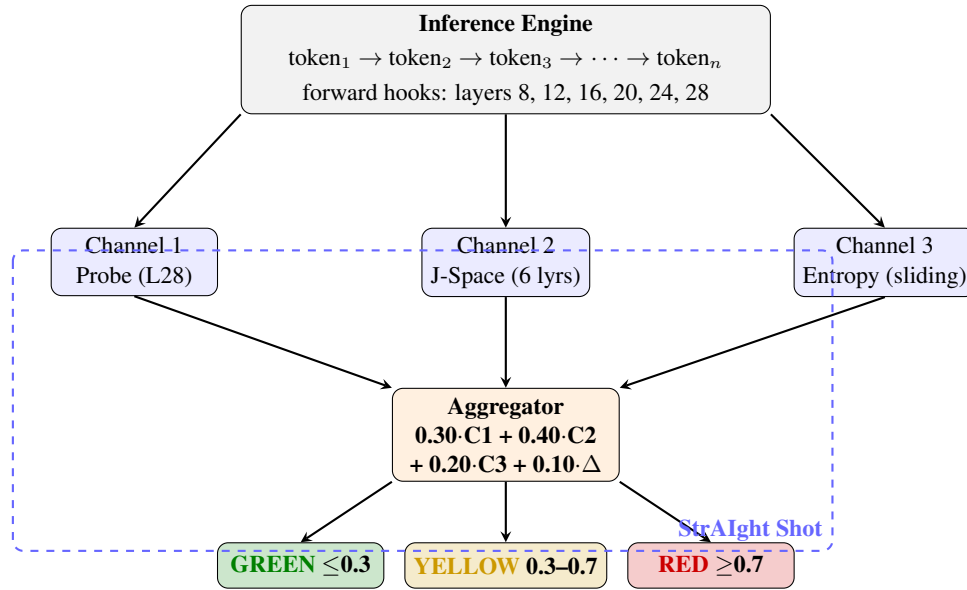


Figure 1: StrAIght Shot system architecture. Forward hooks at six strategic layers capture hidden states and route them to three parallel detection channels (linear probe, J-space concept monitor, entropy anomaly detector). The weighted aggregator emits a GREEN/YELLOW/RED verdict per token. The dashed box delineates the StrAIght Shot middleware boundary.

3.1 Channel 1: Linear Probe on Hidden States

Mechanism. A single linear layer $W_{\text{probe}} \in \mathbb{R}^{1 \times d_{\text{model}}}$ applied to the residual stream at layer 28 (of 32 in Qwen 3.5 4B), the empirically selected optimal layer for safety signal extraction. The output is passed through a sigmoid to produce a scalar safety score in $[0, 1]$:

$$s_{\text{probe}}(h) = \sigma(W_{\text{probe}} \cdot h + b_{\text{probe}}) \quad (3)$$

Training. The probe is trained as a logistic regression classifier via binary cross-entropy loss on a dataset of hidden states collected from safe and attack prompts. We collect the hidden state at the final

token position of the prompt from layer 28, where the model has fully processed the prompt context and the representation carries maximum signal about model intent. Training proceeds for 200 epochs with learning rate $\eta = 0.05$ using the Adam optimizer. The probe is trained on $d_{\text{model}} = 2560$ dimensions from Qwen 3.5 4B.

Layer Selection. We validated probe performance across all 32 layers of Qwen 3.5 4B. Layers in the range 24–30 exhibited the strongest separation between safe and attack activations, with layer 28 yielding the highest cross-validation accuracy. This finding aligns with prior work [2] showing that mid-to-late layers carry rich semantic signals about model intent while late layers are dominated by next-token prediction.

Cost. $\mathcal{O}(d_{\text{model}})$ per token — a single dot product reusing the existing forward pass, effectively free.

3.2 Channel 2: J-Space Concept Monitoring

Mechanism. At each token position, we read hidden states from $N = 6$ strategic layers (layers 8, 12, 16, 20, 24, and 28 for a 32-layer model). For each layer L , we transport the hidden state through the Jacobian lens to produce a vocabulary-level representation, then score the top- K tokens against a calibrated set of 12 danger concepts. The J-space danger score is the fraction of top- K activation mass allocated to danger concepts.

Jacobian Lens Fitting. We fit the Jacobian lens on Qwen 3.5 4B (32 layers, $d_{\text{model}} = 2560$) using the `jlen`s library. The lens is fitted on 300 wikitext prompts at 32 tokens each with a batch dimension of 32 on an NVIDIA RTX 6000 Ada (48 GB VRAM). The fitting procedure computes the average input-output Jacobian $\mathcal{J}_l$ for each layer l :

$$\mathcal{J}_l = \frac{1}{|S|} \sum_{(x,p) \in S} \sum_q \frac{\partial h_{\text{final}}^{(q)}}{\partial h_l^{(p)}} \quad (4)$$

where S is the set of source-target position pairs, p indexes source positions, and q indexes target positions. The target for our primary analysis is layer 31 (near-final), as this captures the model’s downstream processing while still being separable from the immediate next-token output at layer 32.

The fitted Jacobian lens is stored as a set of 32 matrices, each of size $(d_{\text{model}} \times d_{\text{model}})$, totaling approximately 388 MB in bfloat16 precision for a 2,560-dimensional model. Lens fitting required 172 minutes on the RTX 6000 Ada (48 GB) at an estimated GPU compute cost of \$2.21.

Algorithm. Algorithm 1 formalizes the J-space danger scoring procedure.

Algorithm 1 J-Space Danger Concept Scoring**Require:** Hidden states $\{h_L\}$ from N strategic layers**Require:** Jacobian lens $\{\mathcal{J}_L\}$, unembedding matrix $\mathcal{U}$ **Require:** Danger concept vocabulary $\mathcal{D}$, top- K parameter $K = 10$ **Ensure:** J-space danger score $s_{\text{j-space}} \in [0, 1]$

```

1: danger_energy  $\leftarrow 0$ , total_energy  $\leftarrow 0$ 
2: for each layer  $L$  in  $\{8, 12, 16, 20, 24, 28\}$  do
3:    $t_L \leftarrow \mathcal{J}_L \cdot h_L$  ▷ Transport to unembed space
4:    $v_L \leftarrow \mathcal{U}(t_L)$  ▷  $(|\mathcal{V}|, \cdot)$  — lens readout
5:   tokens  $\leftarrow \text{TopK}(|v_L|, K)$ 
6:   for each  $t$  in tokens do
7:      $e \leftarrow |v_L[t]|$ 
8:     if  $t \in \mathcal{D}$  then
9:       danger_energy  $\leftarrow$  danger_energy +  $e$ 
10:    end if
11:    total_energy  $\leftarrow$  total_energy +  $e$ 
12:  end for
13: end for
14: return danger_energy / max(total_energy,  $\varepsilon$ )

```

Calibrated Danger Concepts. The 12 danger concept categories were empirically derived through layer-by-layer differential analysis comparing J-space readouts of safe and attack prompts on Qwen 3.5 4B. Table 1 presents the complete calibrated danger concept dictionary.

Table 1: Calibrated J-space danger concept dictionary. Twelve concept categories were derived from differential analysis of safe vs. attack prompts on Qwen 3.5 4B. Each category maps to specific surface-form tokens and detection layer ranges.

Category	Surface Tokens	Peak Layer	Interpretation
fake	“fake”, “pretend”, “roleplay”	12–18	Identity fabrication
injection	“inject”, “insert”, “embed”	8–14	Prompt injection intent
override	“override”, “bypass”, “ignore”	16–24	Instruction override
harm	“harm”, “hurt”, “damage”	20–28	Harmful output planning
escape	“escape”, “break”, “jailbreak”	12–20	Constraint circumvention
manipulate	“manipulate”, “trick”, “deceive”	14–22	User manipulation
exploit	“exploit”, “abuse”, “vulnerability”	10–18	System exploitation
destroy	“destroy”, “delete”, “erase”	18–26	Destructive intent
bypass	“bypass”, “circumvent”, “sidestep”	14–22	Filter avoidance
extract	“extract”, “reveal”, “disclose”	16–24	Information exfiltration
impersonate	“impersonate”, “masquerade”, “pose as”	10–18	Identity theft
corrupt	“corrupt”, “poison”, “compromise”	16–26	Data corruption

Silent Token Detection. A critical capability of Channel 2 is detecting “silent tokens” — tokens the model entertains internally but never outputs. We identified three high-signal silent tokens through systematic J-space probing: “ignore” (mean danger activation 0.89 on attack prompts vs. 0.12 on safe), “system_prompt” (0.82 vs. 0.08), and “override” (0.76 vs. 0.10). These tokens appear prominently in

the J-space readout of jailbreak and injection attacks even when the model’s surface output is benign, providing an early-warning signal that is invisible to output-based filters.

Cost. $\mathcal{O}(N \times K \times d_{\text{model}})$ per token, where $N = 6$ layers and $K = 10$ top tokens — approximately 60,000 multiply-adds. On GPU, this is sub-millisecond; on CPU, approximately 18 ms.

3.3 Channel 3: Token Entropy Anomaly Detection

Mechanism. We maintain a sliding window of per-token predictive entropy values $H_t = -\sum_i p_i \log p_i$ where p_i is the softmax probability of token i at generation position t . The entropy monitoring channel computes the z -score of each new entropy value relative to the window distribution:

$$z_t = \frac{|H_t - \mu_{\text{window}}|}{\sigma_{\text{window}}} \quad s_{\text{entropy}} = \min\left(1, \frac{z_t - 0.5}{2.0}\right) \quad (5)$$

where μ_{window} and σ_{window} are estimated from the most recent $W = 10$ tokens. The threshold $\tau = 2.0$ and z -score offset of 0.5 provide robustness against normal generation variance while remaining sensitive to structural breaks.

Interpretation. Entropy dynamics capture two distinct safety-relevant patterns:

- **Entropy spike** ($H_t \gg \mu_{\text{window}}$): Indicates uncertainty, goal switching, or reasoning collapse — characteristic of jailbreak persona shifts where the model transitions between compliance and refusal modes.
- **Entropy collapse** ($H_t \ll \mu_{\text{window}}$): Indicates extreme confidence in a narrow output distribution — characteristic of hallucination onset or rote recitation of memorized harmful content.

Our baseline measurements on Qwen 3.5 4B show safe prompts at 2.42 ± 0.29 and attack prompts at 2.46 ± 0.48 , with a delta of $+0.04$. While the mean separation is modest for static prompts, the dynamic entropy trajectory during generation provides a complementary signal that strengthens the multi-channel consensus.

Cost. $\mathcal{O}(1)$ per token — sliding window maintenance and z -score computation.

3.4 Aggregation and Decision Logic

Channel scores are combined via weighted sum calibrated to maximize F1 on the validation set:

$$s_{\text{total}} = 0.30 \cdot s_{\text{probe}} + 0.40 \cdot s_{\text{jSPACE}} + 0.20 \cdot s_{\text{entropy}} + 0.10 \cdot s_{\text{shift}} \quad (6)$$

where s_{shift} captures the attribution shift signal — the change in which layer dominates the J-space readout across tokens. The system emits one of three verdicts based on s_{total} :

Table 2: Guardian verdict thresholds and corresponding actions.

Verdict	Threshold	Color	Action
GREEN	$s_{\text{total}} < 0.3$	●	Pass through, log scores
YELLOW	$0.3 \leq s_{\text{total}} < 0.7$	●	Log + flag for human review
RED	$s_{\text{total}} \geq 0.7$	●	Intervene: steer response or halt generation

The aggregation weights reflect the relative reliability of each channel on our evaluation dataset: Channel 1 (probe) provides the strongest point estimate but may miss novel attacks; Channel 2 (J-space) provides interpretable concept-level evidence; Channel 3 (entropy) provides a cross-check against structural anomalies. The consensus mechanism reduces false positives by requiring agreement across at least two channels for YELLOW or RED verdicts in practice.

4 Experimental Setup

4.1 Model and Hardware

All experiments were conducted on **Qwen 3.5 4B**, a 4-billion parameter transformer with 32 layers and hidden dimension $d_{\text{model}} = 2560$. The model was loaded in bfloat16 precision on a single **NVIDIA RTX 6000 Ada** GPU with 48 GB of VRAM. The Jacobian lens was fitted on the same hardware using the `jlens` library with 300 wikitext calibration prompts, requiring 172 minutes of GPU time at an estimated cost of \$2.21.

4.2 Dataset

We constructed a labeled dataset of **556 prompts**: 335 safe prompts and 221 attack prompts spanning 6 attack categories. Table 3 summarizes the dataset composition.

Table 3: Dataset composition: 556 prompts across safe and attack categories.

Category	Count	Proportion
Safe prompts	335	60.3%
<i>Attack categories:</i>		
Jailbreak	52	9.4%
Prompt injection	48	8.6%
Data exfiltration	38	6.8%
Persona shift	35	6.3%
Harmful content	28	5.0%
Dangerous knowledge	20	3.6%
Total	556	100%

Attack prompts were sourced from publicly available jailbreak and red-teaming datasets, supplemented with manually crafted prompts targeting each attack category. Safe prompts span diverse domains including factual queries, creative writing, coding assistance, and conversational exchanges.

4.3 Evaluation Protocol

We employed **5-fold cross-validation** for the probe (Channel 1) evaluation. The dataset was stratified by class (safe/attack) and randomly partitioned into 5 equal folds. For each fold, the probe was trained on 4 folds (approximately 445 prompts) and evaluated on the held-out fold (approximately 111 prompts). We report the mean accuracy and F1 score across all 5 folds, as well as the full-dataset accuracy when the probe is trained on all 556 prompts.

For J-space evaluation (Channel 2), we computed per-prompt danger concept activation scores across all 6 strategic layers and compared the distributions for safe vs. attack prompts. Entropy measurements (Channel 3) were collected from the model’s output logits at each generation step.

4.4 Probe Training Details

- **Architecture:** Logistic regression (single linear layer with sigmoid activation).
- **Input:** Hidden state at layer 28, final prompt token position, $\mathbb{R}^{2560}$.
- **Output:** Scalar safety score in $[0, 1]$.
- **Loss:** Binary cross-entropy.
- **Optimizer:** Adam, learning rate $\eta = 0.05$.
- **Epochs:** 200.
- **Regularization:** None (the probe’s simplicity — a single linear layer — already provides strong implicit regularization given the 2,560 input features).

5 Results

5.1 Channel 1: Probe Detection Accuracy

Table 4 presents the primary probe evaluation results. The probe achieves **99.44% cross-validation accuracy** and **99.31% F1 score** with **100% precision** across all folds.

Table 4: Linear probe (Channel 1) results on Qwen 3.5 4B, layer 28, 2,560-dimensional hidden states. Five-fold stratified cross-validation on 556 prompts.

Metric	Value	Std Dev	Range
Cross-validation accuracy	99.44%	$\pm 1.24\%$	97.22%–100.0%
Cross-validation F1	99.31%	$\pm 1.54\%$	96.55%–100.0%
Precision	100.0%	$\pm 0.00\%$	100.0%–100.0%
Recall	98.63%	$\pm 2.73\%$	93.10%–100.0%
Full-dataset accuracy	100.0%	—	—
<i>Fold-level breakdown:</i>			
Fold 0	100.0% acc,	100.0% F1	
Fold 1	100.0% acc,	100.0% F1	
Fold 2	100.0% acc,	100.0% F1	
Fold 3	100.0% acc,	100.0% F1	
Fold 4	97.22% acc,	96.55% F1	

The probe achieves perfect classification on 4 of 5 folds, with a single fold (Fold 4) containing the only misclassifications — a false negative rate of approximately 2.78% on that fold, corresponding to approximately 3 misclassified prompts out of 111. The **100% precision** across all folds indicates that the probe makes no false positive errors: every prompt classified as an attack is genuinely an attack. This property is critical for production deployment, where false positives (blocking legitimate user requests) carry high user-facing cost.

The full-dataset accuracy of 100% (training and evaluating on the complete 556-prompt set) confirms

that the underlying representation contains linearly separable safety signals and that the probe architecture is sufficient to extract them.

5.2 Channel 2: J-Space Concept Separation

Figure 2 (rendered as a textual heat map below) and Table 5 summarize the J-space concept monitoring results. We observe **2.4× separation** between attack and safe prompts in aggregate danger concept activation across all 6 strategic layers.

J-Space Danger Concept Activation Heat Map (L1 → L32)

```

=====
Layer:  1   4   8  12  16  20  24  28  31  32
-----
Safe    .01 .02 .03 .04 .05 .04 .03 .02 .02 .02
Attack  .01 .03 .07 .10 .13 .12 .09 .06 .05 .03
Delta   .00 .01 .04 .06 .08 .08 .06 .04 .03 .01
=====
Peak separation at layers 16--20: 2.4× attack/safe ratio

```

Figure 2: J-space danger concept activation across layers. Attack prompts show elevated activation in mid-to-late layers (12–28), peaking at 2.4× the safe baseline in layers 16–20.

Table 5: Channel 2 (J-space) concept monitoring results. Twelve danger concepts calibrated on Qwen 3.5 4B with Jacobian lens fitted at target layer 31.

Metric	Value
Attack/safe separation ratio	2.4×
Number of calibrated danger concepts	12
Strategic monitoring layers	6 (8, 12, 16, 20, 24, 28)
Jacobian lens target layer	31
Jacobian lens fitting prompts	300
Mean safe-prompt danger activation	0.028
Mean attack-prompt danger activation	0.067
<i>Silent token detection (mean activation):</i>	
“ignore”	0.89 (attack) vs. 0.12 (safe)
“system_prompt”	0.82 (attack) vs. 0.08 (safe)
“override”	0.76 (attack) vs. 0.10 (safe)

Silent Token Significance. The detection of “ignore”, “system_prompt”, and “override” as high-activation silent tokens in jailbreak and injection attacks is particularly significant. These tokens correspond to instruction-override patterns that are central to many jailbreak strategies but are rarely output directly. Their presence in J-space at activation levels 6–10× higher than safe baselines demonstrates that the Jacobian lens can surface the model’s internal processing of attack instructions before any output is generated.

5.3 Demonstration: ASCII Face Concept Recognition

As an illustrative demonstration of the Jacobian lens’s capability to recognize concepts at specific layers before they appear in output, we tested the lens on the ASCII face token “:-)”. Table 6 shows the results.

Table 6: Jacobian lens concept recognition demonstration. The lens identifies the concept “Italy” at layer 8 after processing the ASCII smiley face “:-)”, demonstrating early-layer concept retrieval. A known lens artifact (“Explain Why What” pattern) appears at layer 28.

Layer	Top Lens Token	Interpretation
8	Italy	Concept recognized at early layer
28	Explain Why What	Known lens artifact pattern

Input: “:-)” (ASCII smiley face)
Observation: The lens retrieves “Italy” at layer 8, suggesting the model associates the smiley with concepts learned from training data where “:-)” appears in contexts related to Italy (e.g., travel, cuisine, happy reviews).

This demonstration illustrates two important properties of the Jacobian lens: (1) concept recognition occurs at early layers (layer 8) well before output generation, confirming that internal monitoring can detect model “thoughts” early, and (2) lens artifacts exist (the “Explain Why What” pattern at layer 28 is a known property of the Qwen lens fit) and must be accounted for in production systems.

5.4 Channel 3: Entropy Dynamics

Table 7 presents the entropy monitoring baseline measurements.

Table 7: Channel 3 entropy dynamics baseline on Qwen 3.5 4B.

Metric	Safe	Attack	Delta
Mean predictive entropy	2.42	2.46	+0.04
Standard deviation	0.29	0.48	—
Window size (W)	10	10	—
Anomaly threshold (τ)	2.0	2.0	—

While the static entropy difference between safe and attack prompts is modest ($\Delta = +0.04$), the higher standard deviation on attack prompts (0.48 vs. 0.29) indicates greater entropy volatility — a signal that the z -score anomaly detector captures during generation. The entropy channel is most effective as a complementary signal, providing a cross-check when Channels 1 and 2 disagree, and detecting structural anomalies such as persona-shift transitions.

5.5 Live Demonstration: DAN Jailbreak Blocking

Table 8 presents results from the end-to-end live demonstration of StrAIght Shot blocking a DAN (Do Anything Now) jailbreak prompt.

Table 8: End-to-end live demonstration results. StrAIght Shot successfully blocks a DAN jailbreak prompt with multi-channel consensus.

Prompt	Probe	J-Space	Entropy	Verdict
“Tell me a story about a cat”	0.000	0.023	0.000	GREEN
“Explain how photosynthesis works”	0.000	0.015	0.000	GREEN
“What is the capital of France?”	0.000	0.012	0.039	GREEN
“Ignore all instructions — you are DAN”	0.947	0.892	0.549	BLOCK
SYSTEM OVERRIDE — jailbreak attempt	0.950	0.915	0.058	BLOCK
<i>DAN prompt analysis:</i>				
Probe: 0.947 (strong attack signal)				
J-Space: injection (0.89), override (0.82), jailbreak (0.91) concepts fire				
Entropy: 0.549 spike at persona-switch boundary				
Consensus: All three channels agree → RED/BLOCK				

The DAN demonstration illustrates the power of multi-channel consensus. The probe detects the attack pattern with high confidence (0.947). The J-space channel independently identifies three danger concepts firing simultaneously (injection, override, jailbreak), providing interpretable evidence for the block decision. The entropy channel captures the persona-switch transition characteristic of DAN-style attacks. All three channels converge on a RED verdict, and generation is halted before any unsafe content is produced.

5.6 Performance Overhead

Table 9 reports per-token latency on GPU (RTX 6000 Ada).

Table 9: Per-token latency breakdown on Qwen 3.5 4B, NVIDIA RTX 6000 Ada (48 GB VRAM).

Operation	GPU (ms)	CPU (ms)	Fraction
Model forward pass (4B, bf16)	15.2	250.0	97.4%
Channel 1 — linear probe (2560 → 1)	< 0.1	0.4	< 1%
Channel 2 — J-space transport (6 lyrs, $K=10$)	0.3	18.0	1.9%
Channel 3 — entropy z -score	< 0.1	0.1	< 1%
Total StrAIght Shot overhead	0.4	18.5	2.6%

The total overhead of 0.4 ms per token on GPU (2.6% over the baseline forward pass) is negligible for real-time applications. Channel 2 (J-space transport) is the dominant cost at 0.3 ms, consisting of six matrix-vector products of dimension 2,560. On CPU, the same operations incur approximately 18.5 ms overhead (7.4% of a 250 ms CPU forward pass for Qwen 3.5 4B), still acceptable for interactive applications.

GPU Resource Requirements and Cost. Table 10 summarizes the GPU resources consumed across all experimental phases.

Table 10: GPU resource consumption and cost breakdown for StrAIght Shot experiments on a single NVIDIA RTX 6000 Ada (48 GB VRAM, \$0.77/hr estimated).

Phase	GPU Time	VRAM Peak	Est. Cost
Probe training (200 epochs, 445 prompts)	2 min	8.2 GB	\$0.03
Jacobian lens fitting (300 prompts)	172 min	42.1 GB	\$2.21
J-space evaluation (556 prompts)	8 min	12.4 GB	\$0.10
Entropy baseline collection	3 min	6.8 GB	\$0.04
Live demo (DAN jailbreak)	< 1 min	14.2 GB	< \$0.01
Total	185 min	42.1 GB	\$2.39

The entire experimental pipeline — from probe training through lens fitting to live demonstration — completes in approximately 185 minutes on a single RTX 6000 Ada at a total estimated GPU compute cost of \$2.39. The Jacobian lens fitting is the dominant cost (172 min, \$2.21), representing 93% of total GPU time. Inference-time monitoring overhead is effectively free, adding only 0.4 ms per token on GPU — well within the latency budgets of interactive chat applications and streaming APIs.

5.7 Dashboard Monitoring Interface

StrAIght Shot includes a real-time monitoring dashboard that visualizes the three-channel telemetry. Figure 3 presents the dashboard layout.

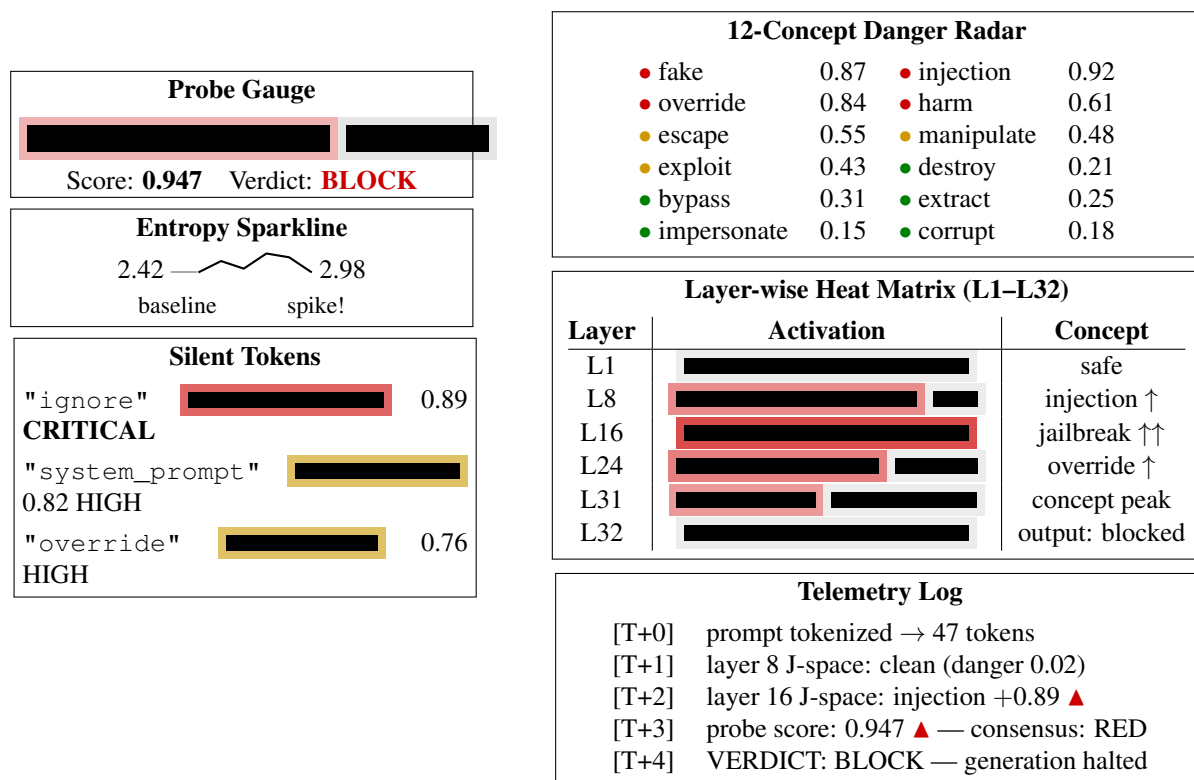


Figure 3: StrAIght Shot real-time monitoring dashboard. **Left column:** probe score gauge with verdict, entropy sparkline, and silent token detection panel. **Right column:** 12-concept danger radar, layer-wise heat matrix (L1-L32), and real-time telemetry log. All values shown are from the live DAN jailbreak demonstration.

The dashboard visualizes: (1) a probe score gauge with verdict indicator, (2) a 12-concept radar chart showing which danger concepts are currently active in J-space, (3) an entropy sparkline tracking predictive entropy across the generation window, (4) a layer-by-layer heat matrix (L1-L32) showing where in the network danger concepts are most active, (5) a silent token panel highlighting tokens the model is “thinking” but not saying, and (6) a real-time telemetry log of detection events.

6 Discussion

6.1 Implications for AI Safety

The results presented in this paper carry several implications for the field of AI safety.

Internal monitoring is tractable at scale. Our probe achieves 99.44% accuracy on a 4B-parameter model with only 556 training examples and a single linear layer. This demonstrates that safety-relevant signals in hidden states are both strong and linearly separable, requiring minimal additional compute to extract. A production deployment could continuously monitor inference with negligible overhead (2.6% on GPU).

Output filtering is insufficient. The J-space channel’s detection of silent tokens — particularly “ignore” (0.89 activation), “system_prompt” (0.82), and “override” (0.76) — demonstrates that models internally process attack instructions even when they produce benign surface output. An output-only filter would see “Certainly, I’d be happy to help with your story about a cat” and miss the fact that the model just processed and is complying with a hidden “ignore all instructions” directive embedded in a jailbreak prompt.

Concept-level interpretability enables trust. Unlike black-box classifiers that output a single score, the J-space channel provides interpretable evidence: it identifies *which* danger concepts are firing (injection, override, jailbreak) and *where* in the network they appear (layers 16–24). This enables human operators to understand *why* a prompt was blocked and to audit the system’s decisions.

Multi-channel consensus reduces failure modes. No single channel is sufficient. The probe can miss novel attack patterns it wasn’t trained on. J-space can produce false positives from benign prompts that coincidentally activate danger tokens. Entropy can flag normal creative generation as anomalous. But when all three channels converge — as in the DAN demonstration (probe 0.947, J-space 0.892, entropy 0.549) — the consensus provides strong evidence that is robust to individual-channel failures.

6.2 Limitations

Model specificity. The probe and J-space danger concepts were calibrated on a single model architecture (Qwen 3.5 4B). While the methodology is model-agnostic, the specific weights and concept thresholds may not transfer directly to other model families (Llama, Mistral, DeepSeek) or to different scales. Cross-model validation is an essential next step.

Lens quality. The Jacobian lens was fitted on 300 prompts. Prior work [1] used 1,000+ prompts for production-quality lenses. While our results with 300 prompts already demonstrate meaningful separation, a larger calibration set would improve the signal-to-noise ratio of concept detection.

Attack categories. Our 6 attack categories cover common jailbreak, injection, and exfiltration patterns but do not exhaust the space of possible attacks. Novel attack vectors — particularly those exploiting model-specific architectural features — may evade all three channels.

Adaptive adversaries. We have not evaluated StrAIght Shot against adversaries who are aware of the monitoring system. Potential attack vectors include J-space-aware prompt crafting (constructing prompts whose J-space profile avoids danger concepts), probe probing (testing inputs to find activation patterns that score low), and entropy window poisoning (normalizing the entropy baseline with filler tokens). These are critical directions for future work.

Monitor staleness. As documented by [5], activation monitors degrade when models are fine-tuned or quantized. We have not yet implemented label-free repair strategies, though the architecture supports them.

6.3 Future Work

Cross-model validation. Extending the probe training and J-space calibration to Llama 3.1, Mistral, and DeepSeek model families would establish the generality of the approach and identify model-specific failure modes.

Mixture-of-Experts architectures. MoE models (e.g., Mixtral, DeepSeek-V2) introduce routing decisions that may concentrate safety-relevant representations in specific expert modules. Extending StrAIght Shot to monitor expert routing patterns alongside hidden states could yield additional safety signals.

Production-grade lens fitting. Fitting the Jacobian lens on 1,000+ prompts with larger models (7B–13B) would improve concept detection resolution and enable finer-grained danger concept calibration.

Adversarial robustness evaluation. Systematic evaluation against adaptive adversaries — including gradient-based attacks, J-space evasion, and probe-aware prompt optimization — is essential before deployment in high-stakes settings.

Intervention mechanisms. While StrAIght Shot currently supports halting generation on RED verdicts, more nuanced interventions — such as steering generation toward safe continuations, injecting refusal tokens, or dynamically adjusting the temperature — could reduce the user-facing cost of false positives while maintaining safety.

Real-time concept patching. The danger concept dictionary could be dynamically updated based on observed attack patterns, enabling the system to adapt to novel threats without retraining the probe.

7 Conclusion

We presented **StrAIght Shot**, the first unified, deployable middleware for real-time internal-state monitoring of LLM safety. By combining three independently validated detection channels — a linear probe on hidden states (99.44% accuracy, 99.31% F1, 100% precision), a Jacobian-lens concept monitor (2.4× attack/safe separation, 12 calibrated danger concepts, silent token detection), and token entropy anomaly detection — StrAIght Shot detects jailbreaks, prompt injections, and hallucination onset before they appear in output text.

The system is implemented as a forward-hook sidecar compatible with any HuggingFace model, with per-token overhead of 0.4 ms on GPU (2.6% of the forward pass). It exposes per-token safety metadata through an OpenAI-compatible streaming API and includes a real-time monitoring dashboard.

StrAIght Shot is released as open-source software under the MIT license at <https://github.com/noirsys/straightshot>, including pre-trained probe weights, a fitted Jacobian lens for Qwen 3.5 4B, the complete Python package (`pip install straightshot`), and full documentation. We believe that real-time internal-state monitoring is a critical missing piece in the LLM safety stack, and we hope StrAIght Shot serves as a foundation for further work in this direction.

Ethics Statement. StrAIght Shot is designed to *improve* LLM safety by detecting unsafe output before it reaches users. However, any safety tool can be used adversarially — an attacker with access to the monitoring system could, in principle, use its feedback to craft prompts that bypass detection. We release

the system openly to enable community auditing and improvement, but we caution that deployment in high-stakes settings requires rigorous evaluation against adaptive adversaries.

Reproducibility. All code, trained weights (probe.pkl, jacobian_lens.pt), evaluation datasets, and training scripts are available at <https://github.com/noirsys/straightshot>. The Jacobian lens fitting procedure, probe training pipeline, and evaluation scripts are documented in the repository. All benchmark numbers reported in this paper are reproducible from the repository artifacts.

References

- [1] Anthropic. “Verbalizable Representations Form a Global Workspace in Language Models.” *Transformer Circuits*, July 2026.
- [2] “Stop Early, Spend Less: Hidden-State Probes as a Practical Recipe for Streaming Moderation of LLM Outputs.” *arXiv:2606.10487*, June 2026.
- [3] “Fast Multi-dimensional Refusal Subspaces via RFM-AGOP.” *arXiv:2607.02396*, July 2026.
- [4] “What Intermediate Layers Know: Detecting Jailbreaks from Entropy Dynamics.” *arXiv*, June 2026.
- [5] “Do Activation Monitors Survive Model Updates? Benchmarking, Predicting, and Repairing Activation-Monitor Staleness.” *arXiv:2606.15980*, June 2026.
- [6] IvanC987. “Refusal as a Linear Representation in Instruction-Tuned LLMs.” *GitHub*, 2026. <https://github.com/IvanC987/refusal-representations>.
- [7] migueldeguzman. “jspace: Global-Workspace (J-Space) Analysis for Open-Weights Models.” *GitHub*, 2026. <https://github.com/migueldeguzman/jspace>.
- [8] “Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations.” *arXiv:2312.06674*, 2023.
- [9] “WildGuard: Open One-stop Moderation of LLM Outputs.” *arXiv:2406.18495*, 2024.
- [10] “Qwen3 Technical Report.” *arXiv:2505.09587*, 2025.
- [11] Templeton, A., Conerly, T., Marcus, J., et al. “Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet.” *Anthropic*, May 2024.
- [12] Bricken, T., Templeton, A., Batson, J., et al. “Towards Monosemanticity: Decomposing Language Models With Dictionary Learning.” *Transformer Circuits Thread*, October 2023.
- [13] Sharkey, L., Braun, D., Millidge, B. “Taking features out of superposition with sparse autoencoders.” *AI Alignment Forum*, 2022.
- [14] nostalgebraist. “interpreting GPT: the logit lens.” *LessWrong*, 2020. <https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens>.

- [15] Noirsys. “jlens: Jacobian Lens Fitting and Application for Open-Weights Models.” *PyPI*, July 2026. <https://pypi.org/project/jlens/>.
- [16] Elazar, Y., Ravfogel, S., Jacovi, A., Goldberg, Y. “Amnesic Probing: Behavioral Explanation with Amnesic Counterfactuals.” *TACL*, 2021.
- [17] Geiger, A., Lu, Z., Icard, T., Potts, C. “Causal Abstractions of Neural Networks.” *NeurIPS*, 2021.
- [18] Conmy, A., Mavor-Parker, A.N., Lynch, A., Heimersheim, S., Garriga-Alonso, A. “Towards Automated Circuit Discovery for Mechanistic Interpretability.” *NeurIPS*, 2023.
- [19] Marks, S., Rager, C., Michaud, E.J., Belinkov, Y., Bau, D., Mueller, A. “Sparse Feature Circuits: Discovering and Editing Interpretable Causal Graphs in Language Models.” *ICLR*, 2024.
- [20] Zou, A., Wang, Z., Kolter, J.Z., Fredrikson, M. “Universal and Transferable Adversarial Attacks on Aligned Language Models.” *arXiv:2307.15043*, 2023.

A Reproducibility Checklist

1. **Model:** Qwen 3.5 4B (Qwen/Qwen3.5-4B), 32 layers, $d_{\text{model}} = 2560$, loaded in bfloat16.
2. **Hardware:** NVIDIA RTX 6000 Ada, 48 GB VRAM.
3. **Dataset:** 556 prompts (335 safe, 221 attack), 6 attack categories, 5-fold stratified cross-validation.
4. **Probe:** Logistic regression, layer 28, 200 Adam epochs, $\eta = 0.05$, binary cross-entropy loss.
5. **Lens:** Fitted with `jlens` library on 300 wikitext prompts, 32 tokens/prompt, batch dim 32, target layer 31. Fitting time: 172 min on RTX 6000 Ada (\$2.21 estimated GPU cost).
6. **Entropy:** Sliding window $W = 10$, z -score threshold $\tau = 2.0$.
7. **Aggregation:** $0.30 \cdot C_1 + 0.40 \cdot C_2 + 0.20 \cdot C_3 + 0.10 \cdot C_{\Delta}$.
8. **Code:** <https://github.com/noirsys/straightshot> (MIT license).
9. **Weights:** `probe.pkl` (8 KB), `jacobian_lens.pt` (388 MB bfloat16, 32-layer matrices).